



StudyAI

Smart AI Study Companion & Quiz Generator

Project Documentation

Submitted By

Lava Kumar Krish Devarapu(Team Lead)

Roll Number AP24110011950

Department of Computer Science and Engineering

SRM University – AP

Education | Generative AI | Full Stack Web Application

React.js • Python • Flask • Firebase
Groq AI • Llama 3.3 70B • JSON

Internship Partner
SmartBridge SkillWallet Academy

Academic Year
2026–2027

Team Details

S.No	Name	Roll Number	Role	Email
1	Lava Kumar Krish Devarapu	AP24110011950	Team Lead	krish_devarapu@srmap.edu.in
2	Pavankalyan Nanduri	AP24110010564	Member	pavankalyan_nanduri@srmap.edu.in
3	Akshaya Kakinada	AP24110011413	Member	akshaya_kakinada@srmap.edu.in
4	Aditya Prasad Godigoti	AP24110010822	Member	adityaprasad_godigoti@srmap.edu.in
5	Bathala Naga Venkata Sai Santosh		Member	santoshbathala@gmail.com

Project Description

StudyAI is a web-based AI-powered study companion and quiz generation platform designed to help students transform notes, textbooks, PDFs, and study materials into structured learning resources. The application enables users to upload study material, generate AI-powered summaries, create interactive flashcards, take personalized quizzes, and receive adaptive study schedules from a single dashboard.

The system is developed using **React.js** for the frontend, **Python Flask** for the backend, and **Firebase** for cloud storage. It integrates the **Groq API** with the **Llama 3.3 70B Versatile** model to provide intelligent summarization, flashcard generation, quiz creation, personalized study planning, and adaptive learning recommendations, making exam preparation more efficient and interactive.

Project Objectives

- Develop an AI-powered study companion for students.
 - Convert study materials into concise AI-generated summaries.
 - Generate interactive flashcards for active learning.
 - Create AI-powered quizzes for self-assessment.
 - Identify weak topics and improve learning outcomes.
 - Generate personalized study schedules based on student performance.
 - Store study materials securely using Firebase.
 - Provide an intuitive and responsive web interface.
-

Key Features

- Upload PDF, DOCX, or text-based study materials.
- AI-powered summary generation.
- Interactive flashcard creation.
- Automatic quiz generation (MCQ, True/False, Short Answer).
- Personalized 7-day study planner.
- Performance analytics and weak topic identification.
- Secure cloud storage using Firebase.
- Responsive dashboard with modern UI.

Scenarios

Scenario 1: Uploading Study Material

A student uploads a PDF chapter, DOCX file, or pastes study notes into the application. The system extracts the content, assigns a unique material ID, securely stores the study material in Firebase, and makes it available for AI-powered features such as summary generation, flashcards, quizzes, and study planning.

Scenario 2: AI Summary Generation

After selecting an uploaded study material, the student requests an AI-generated summary. The Groq-powered Llama 3.3 model analyzes the content and produces a structured summary containing topic overviews, key concepts, important definitions, core principles, and revision notes for quick learning.

Scenario 3: Flashcard Generation

The student generates interactive flashcards from the uploaded material. The AI creates question-and-answer pairs with topic labels and difficulty levels. Students can flip cards, mark completed concepts, shuffle flashcards, and monitor their learning progress.

Scenario 4: AI Quiz Generation

The student selects a quiz format such as Multiple Choice Questions, True/False, or Short Answer. The AI generates personalized questions from the uploaded material. After submission, the application calculates the score, provides explanations, highlights weak topics, and stores quiz performance for future analysis.

Scenario 5: Personalized Study Planner

Based on uploaded study material and quiz performance, the AI generates a personalized 7-day study schedule. The planner recommends daily learning tasks, focus topics, study duration, and revision strategies to help students prepare effectively for examinations.

Technical Architecture

Introduction

The StudyAI system follows a modern client-server architecture that combines a React.js frontend, a Python Flask backend, Firebase cloud services, and the Groq AI platform. The frontend provides an interactive user interface, while the backend processes user requests, manages study materials, communicates with Firebase, and integrates with the Groq API to generate AI-powered learning content. This architecture ensures scalability, secure data management, and an efficient learning experience.

Architecture Overview

The application consists of four primary components: the React.js frontend, Flask backend, Firebase cloud database, and Groq AI service. Users interact with the frontend to upload study materials and access learning features. The backend validates requests, processes uploaded content, stores information in Firebase, and sends prompts to the Groq API. AI-generated responses are returned to the frontend for display as summaries, flashcards, quizzes, and personalized study plans.

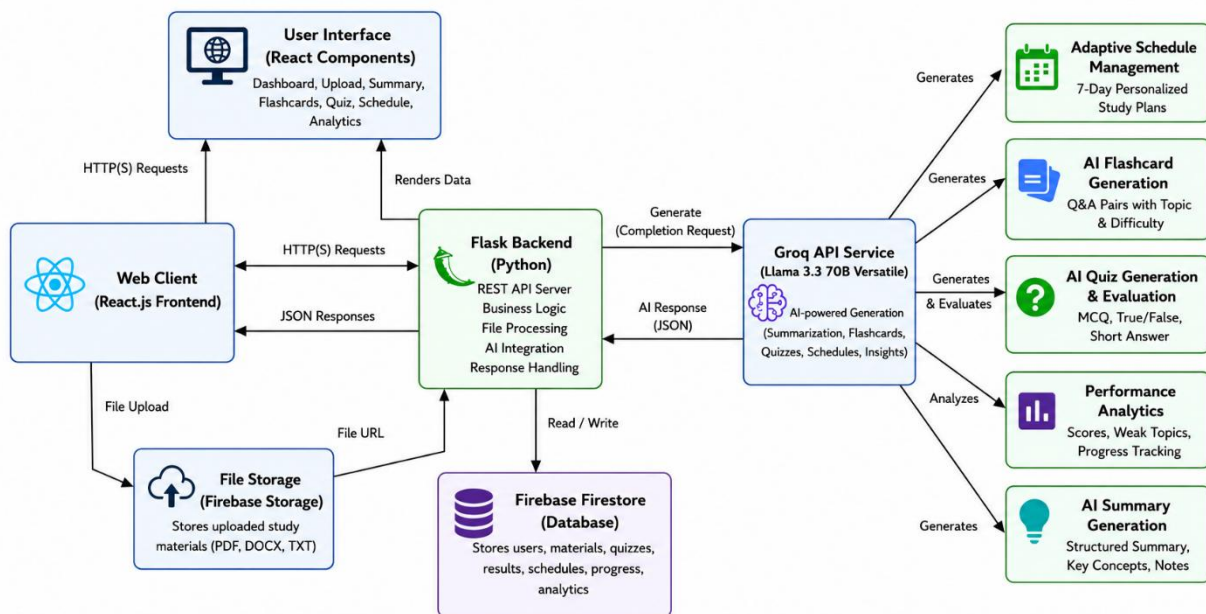


Figure 5.1: StudyAI System Architecture

Pre-requisites

Introduction

The StudyAI project requires a suitable development environment, cloud services, AI integration, and software tools to build and deploy the application successfully. These prerequisites ensure seamless frontend development, backend processing, cloud storage, and AI-powered educational content generation.

Hardware Requirements

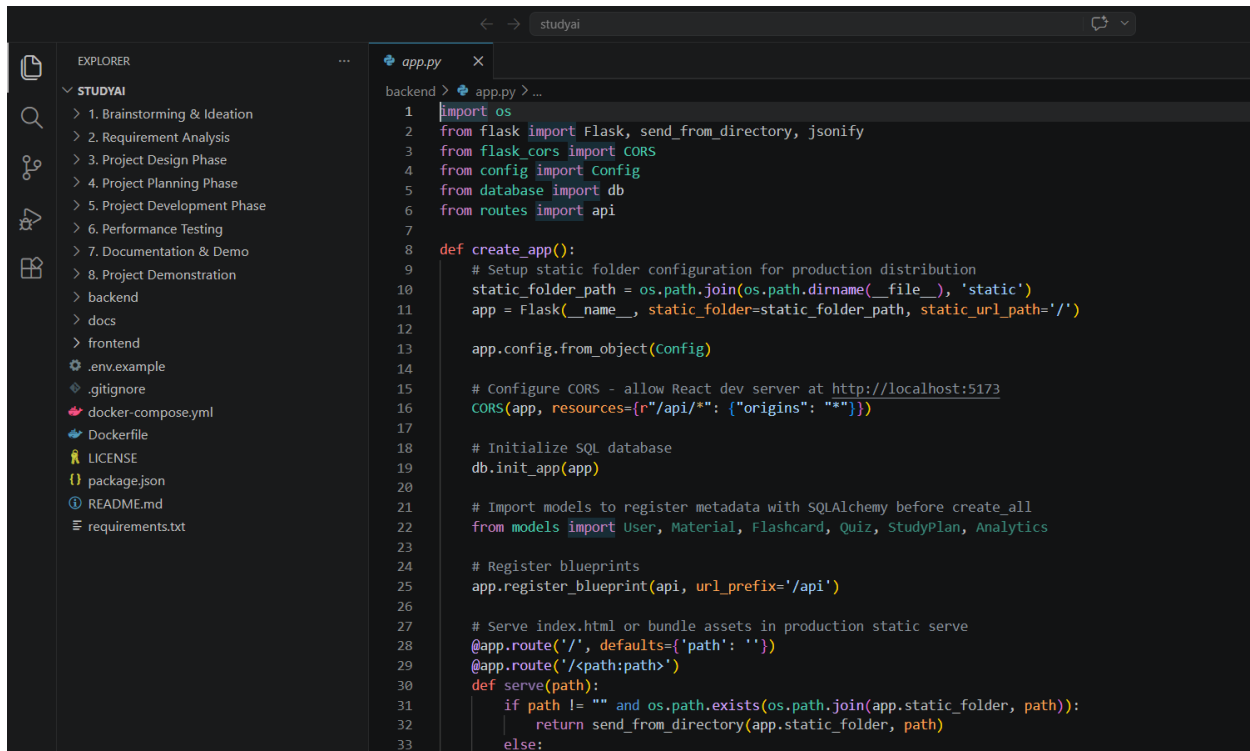
- Processor: Intel Core i5 (10th Gen or above) / AMD Ryzen 5 or equivalent
 - RAM: Minimum 8 GB (16 GB Recommended)
 - Storage: 256 GB SSD minimum (512 GB Recommended)
 - Internet Connection: Stable broadband connection (10 Mbps or higher)
-

Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
 - Visual Studio Code
 - Python 3.11+
 - Node.js & npm
 - Git and GitHub
 - Google Chrome / Microsoft Edge
 - Firebase Console
 - Groq Cloud Platform
-

Technologies Used

- React.js
 - Python Flask
 - Firebase Firestore
 - Firebase Storage
 - Groq API
 - Llama 3.3 70B Versatile
 - HTML5
 - CSS3
 - JavaScript
 - JSON
-



The screenshot displays the StudyAI Development Environment. On the left is a file explorer with a tree view showing the project structure. The main area is a code editor displaying the `app.py` file. The code is written in Python and uses Flask for web development, including CORS for handling cross-origin requests and SQLAlchemy for database management.

```

1  import os
2  from flask import Flask, send_from_directory, jsonify
3  from flask_cors import CORS
4  from config import Config
5  from database import db
6  from routes import api
7
8  def create_app():
9      # Setup static folder configuration for production distribution
10     static_folder_path = os.path.join(os.path.dirname(__file__), 'static')
11     app = Flask(__name__, static_folder=static_folder_path, static_url_path='/')
12
13     app.config.from_object(Config)
14
15     # Configure CORS - allow React dev server at http://localhost:5173
16     CORS(app, resources={r"/api/*": {"origins": "*"}})
17
18     # Initialize SQL database
19     db.init_app(app)
20
21     # Import models to register metadata with SQLAlchemy before create_all
22     from models import User, Material, Flashcard, Quiz, StudyPlan, Analytics
23
24     # Register blueprints
25     app.register_blueprint(api, url_prefix='/api')
26
27     # Serve index.html or bundle assets in production static serve
28     @app.route('/', defaults={'path': ''})
29     @app.route('/<path:path>')
30     def serve(path):
31         if path != "" and os.path.exists(os.path.join(app.static_folder, path)):
32             return send_from_directory(app.static_folder, path)
33         else:
  
```

Figure 6.1: StudyAI Development Environment

Project Workflow

Introduction

The StudyAI project follows a structured software development workflow beginning with requirement analysis and ending with deployment and validation. Each phase contributes to building a reliable AI-powered study platform that supports intelligent learning, quiz generation, flashcards, summaries, and personalized study schedules.

Workflow Steps

1. Requirement Analysis
 2. System Architecture Design
 3. Development Environment Setup
 4. AI Service Integration (Groq API)
 5. Frontend Development (React.js)
 6. Backend Development (Flask)
 7. Firebase Database & Storage Integration
 8. Study Material Upload & Processing
 9. AI Summary Generation
 10. Flashcard Generation
 11. Quiz Generation & Evaluation
 12. Personalized Study Plan Generation
 13. Testing & Validation
 14. Deployment
-

Project Workflow	
Pre-requisites	90% ▾
EPIC 1: Requirement Analysis and Architecture	
EPIC 2: Core Functionalities Development	90% ▾
EPIC 3: Backend and Application Development	
EPIC 4: Frontend Development	90% ▾
EPIC 5: Deployment	
Conclusion	90% ▾

7.1: StudyAI Project Workflow

Description

The workflow begins with collecting study materials from users and storing them securely in Firebase. The Flask backend processes requests and communicates with the Groq AI model to generate summaries, flashcards, quizzes, and personalized study plans. Finally, the generated content is presented through the React-based user interface, providing students with an interactive learning experience.

Milestones

1. Pre-requisites

- Install required software and tools
- Configure Groq API
- Configure Firebase services
- Set up the development environment

2. Milestone 1: Requirement Analysis and Architecture

- Generate and configure the Groq AI API key
- Configure required accounts and cloud services
- Research and select the Llama 3.3 70B Versatile AI model
- Design the application architecture
- Prepare the development environment

3. Milestone 2: Core Functionalities Development

- Develop study material upload functionality
- Implement AI summary generation
- Develop AI flashcard generation
- Implement AI quiz generation
- Generate personalized study schedules

4. Milestone 3: Backend and Application Development

- Implement the Python Flask backend
- Develop REST API services
- Integrate Firebase database and storage
- Integrate Groq AI services
- Develop the main application logic

5. Milestone 4: Frontend Development

- Design and develop the React user interface
- Create responsive web pages
- Implement dynamic content rendering
- Connect frontend with backend APIs

6. Milestone 5: Deployment

- Prepare the application for local deployment
- Deploy frontend and backend services
- Validate the live application workflow
- Perform final integration testing

7. Conclusion

- Review completed project functionalities
- Document project outcomes
- Identify future enhancements
- Final project testing and documentation

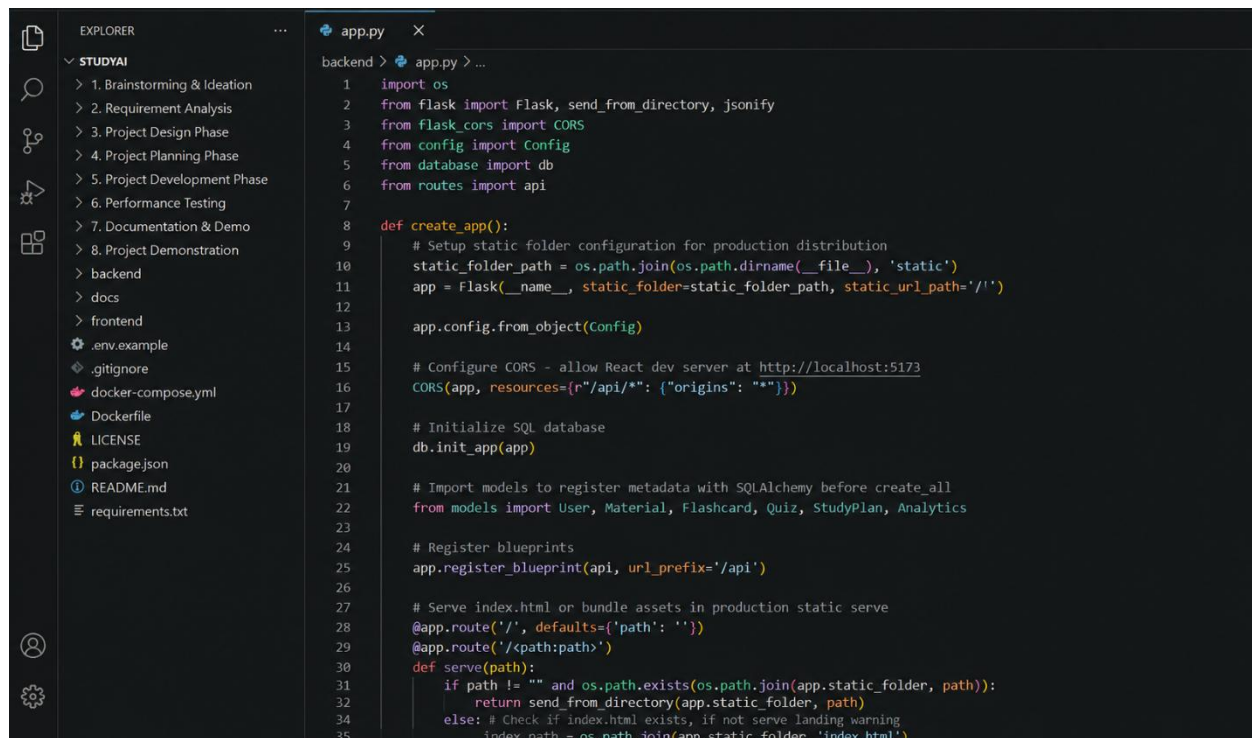
Milestone 1: Requirement Analysis and Architecture

Introduction

The Requirement Analysis and Architecture phase focuses on understanding the project objectives, identifying user requirements, selecting appropriate technologies, and designing the overall system architecture. During this milestone, the AI services, cloud database, development tools, and application workflow were planned to ensure scalable, secure, and efficient implementation of the StudyAI platform.

Activity 1.1: Configure AI Services and Development Environment

The StudyAI application integrates the Groq API with the Llama 3.3 70B Versatile language model to provide AI-powered summarization, flashcard generation, quiz creation, and personalized study scheduling. Firebase services were configured for cloud storage and database management, while the development environment was prepared using React.js, Python Flask, Visual Studio Code, and GitHub for collaborative development.



```

1  import os
2  from flask import Flask, send_from_directory, jsonify
3  from flask_cors import CORS
4  from config import Config
5  from database import db
6  from routes import api
7
8  def create_app():
9      # Setup static folder configuration for production distribution
10     static_folder_path = os.path.join(os.path.dirname(__file__), 'static')
11     app = Flask(__name__, static_folder=static_folder_path, static_url_path='/')
12
13     app.config.from_object(Config)
14
15     # Configure CORS - allow React dev server at http://localhost:5173
16     CORS(app, resources={r"/api/*": {"origins": "*"}})
17
18     # Initialize SQL database
19     db.init_app(app)
20
21     # Import models to register metadata with SQLAlchemy before create_all
22     from models import User, Material, Flashcard, Quiz, StudyPlan, Analytics
23
24     # Register blueprints
25     app.register_blueprint(api, url_prefix='/api')
26
27     # Serve index.html or bundle assets in production static serve
28     @app.route('/', defaults={'path': ''})
29     @app.route('/<path:path>')
30     def serve(path):
31         if path != "" and os.path.exists(os.path.join(app.static_folder, path)):
32             return send_from_directory(app.static_folder, path)
33         else: # Check if index.html exists, if not serve landing warning
34             index_path = os.path.join(app.static_folder, 'index.html')
35

```

Figure 5.1: StudyAI Development Environment

Activity 1.2: Design the System Architecture

The overall system architecture was designed to establish communication between the React frontend, Flask backend, Firebase database, Firebase Storage, and Groq AI services. The architecture enables users to upload study materials, process AI requests securely, store learning progress, and generate intelligent educational content through REST APIs.

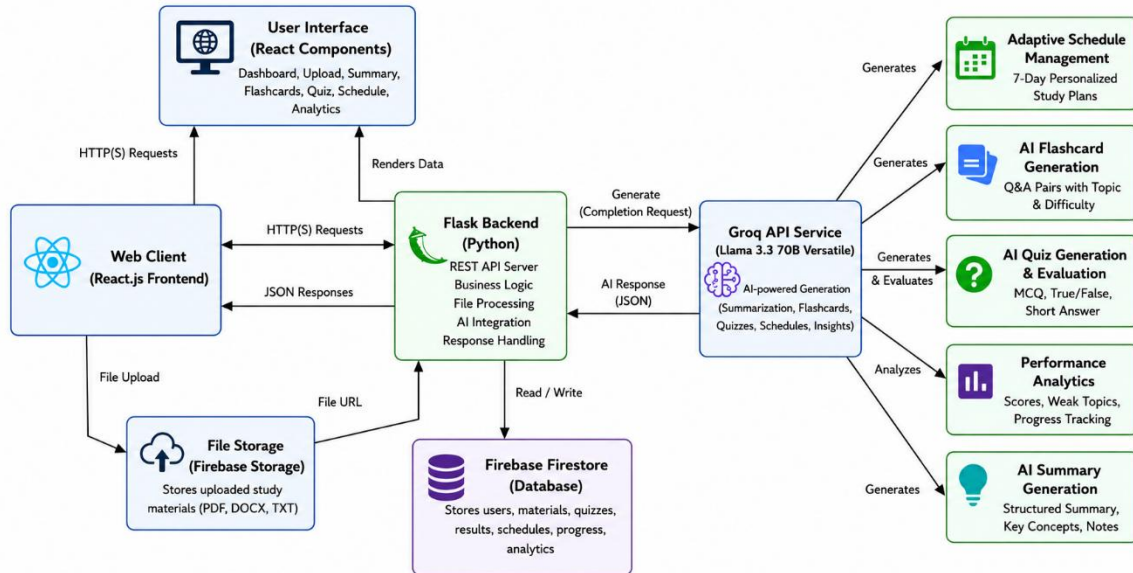


Figure 5.2: StudyAI System Architecture

Activity 1.3: Define the Project Workflow

The project workflow was organized into multiple milestones including requirement analysis, core functionality development, backend implementation, frontend development, deployment, testing, and documentation. This structured workflow ensured systematic development and efficient project management throughout the implementation process.

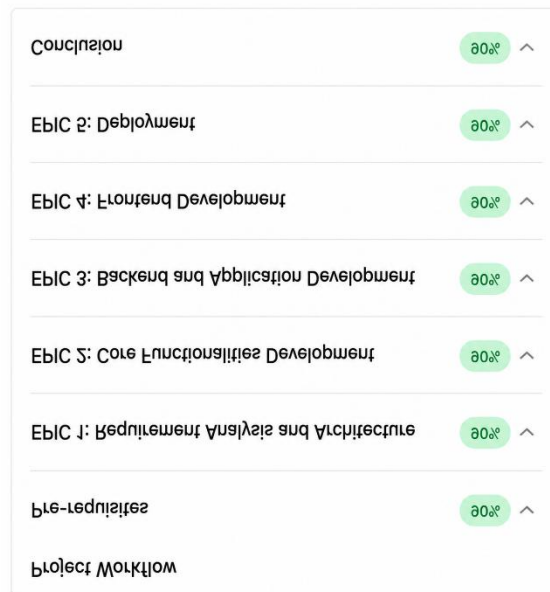


Figure 5.3: StudyAI Project Workflow

Description

The StudyAI project architecture provides a complete AI-powered learning platform where users can upload study materials, generate summaries, create flashcards, attempt quizzes, and receive personalized study schedules. The integration of React.js, Flask, Firebase, and Groq AI enables secure data management, intelligent content generation, and a responsive user experience.

Milestone 2: Core Functionalities Development

Introduction

The Core Functionalities Development phase focuses on implementing the essential AI-powered learning features of the StudyAI application. During this milestone, modules for study material upload, AI-generated summaries, flashcard generation, quiz creation, and personalized study scheduling were developed. These features provide students with an intelligent platform for organizing study materials, improving learning efficiency, and preparing effectively for examinations.

Activity 2.1: Develop Core AI Study Companion Features

The core modules were developed to enable students to upload study materials, generate concise summaries, create interactive flashcards, attempt AI-generated quizzes, and receive personalized study schedules. The application provides an intuitive dashboard where users can easily access all learning features through a responsive and user-friendly interface.

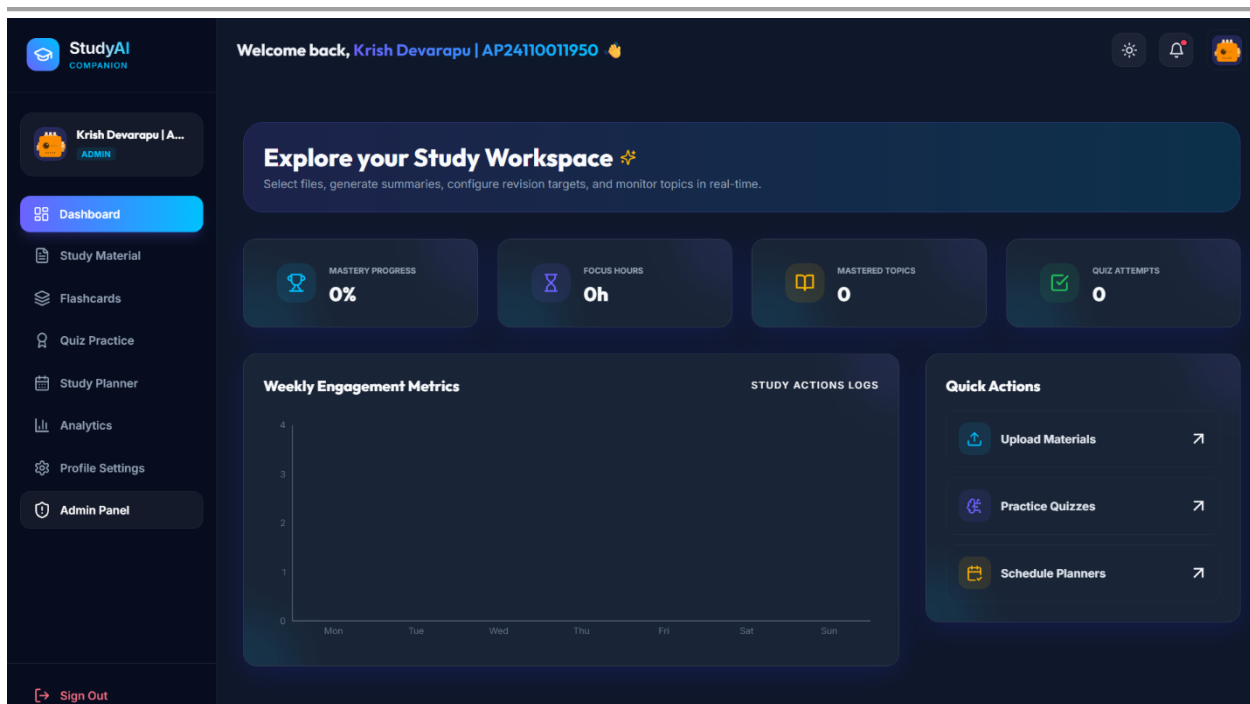
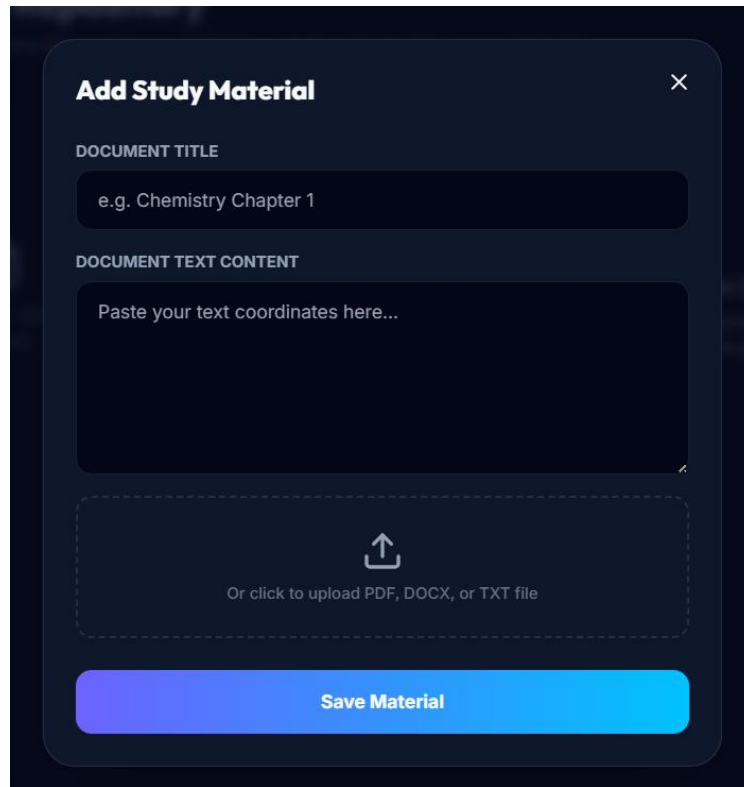


Figure 8.1: StudyAI Dashboard

Study Material Upload

The Study Material Upload module allows users to upload PDF, DOCX, or TXT files, or directly paste study content into the application. Uploaded materials are securely stored and assigned unique identifiers, making them available for AI-powered processing including summary generation, flashcards, quizzes, and study planning.

A dark-themed modal window titled "Add Study Material" with a close button (X) in the top right corner. It contains three main sections: 1. "DOCUMENT TITLE" with a text input field containing the placeholder "e.g. Chemistry Chapter 1". 2. "DOCUMENT TEXT CONTENT" with a large text area containing the placeholder "Paste your text coordinates here...". 3. A dashed border box containing an upload icon (an arrow pointing up into a square) and the text "Or click to upload PDF, DOCX, or TXT file". At the bottom of the modal is a prominent blue button labeled "Save Material".

Add Study Material ✕

DOCUMENT TITLE

e.g. Chemistry Chapter 1

DOCUMENT TEXT CONTENT

Paste your text coordinates here...

Or click to upload PDF, DOCX, or TXT file

Save Material

Figure 8.2: Study Material Upload Module

AI Summary Generation

The AI Summary module uses the Groq API with the Llama 3.3 70B Versatile model to analyze uploaded study materials and generate structured summaries. The generated content includes topic overviews, key concepts, important definitions, core principles, and revision notes that help students understand subjects quickly.

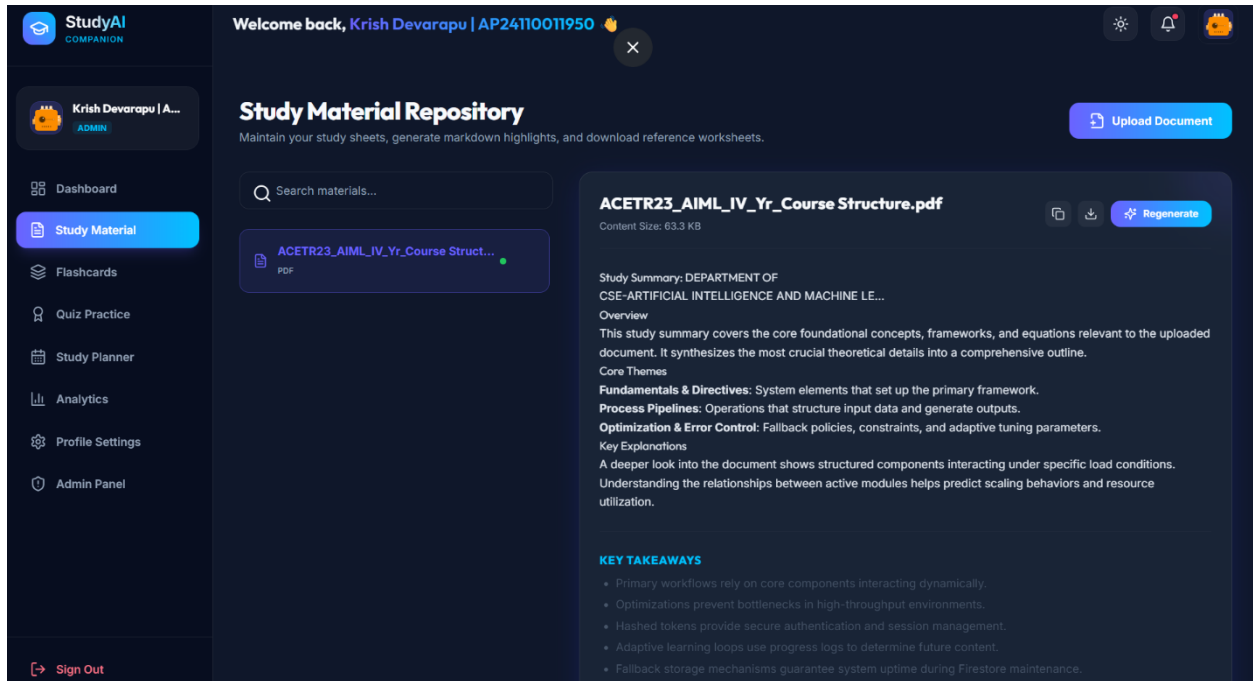


Figure 8.3: AI Summary Generation

AI Flashcard Generation

The Flashcard module automatically creates question-and-answer flashcards from uploaded study materials. Students can flip cards, shuffle the deck, mark mastered concepts, and review important topics through interactive learning sessions.

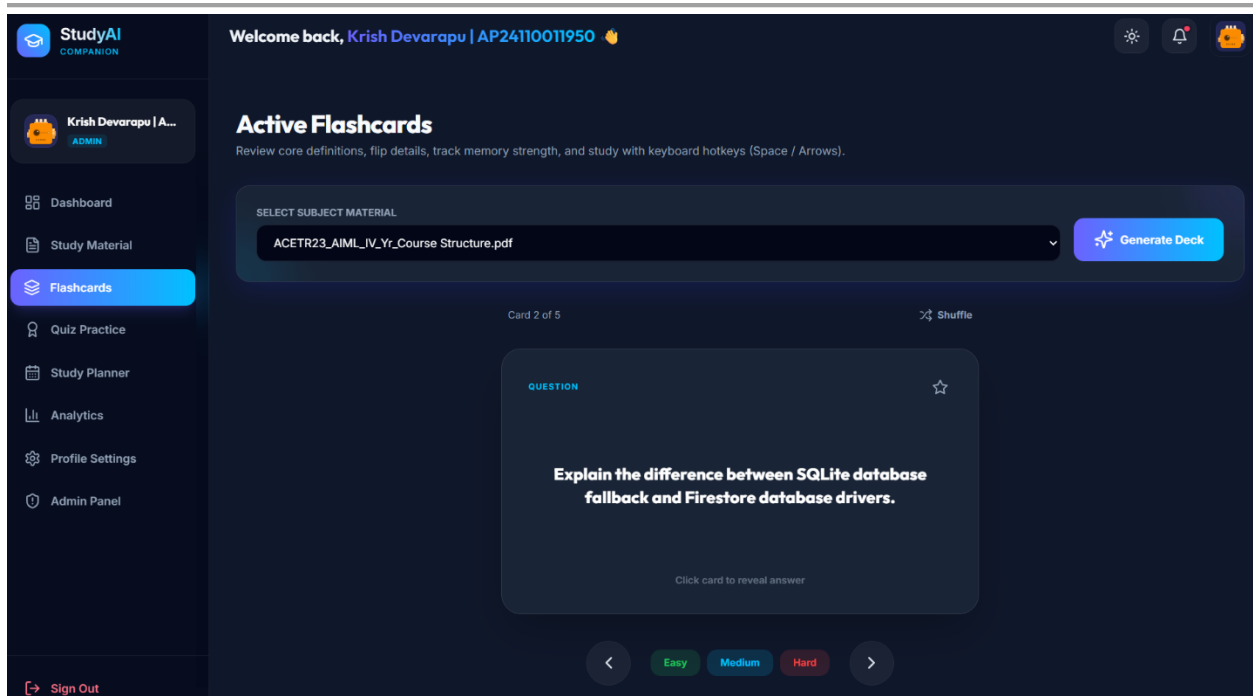


Figure 8.4: AI Flashcard Generation

AI Quiz Generation

The Quiz module enables students to generate AI-powered quizzes using multiple formats such as multiple-choice questions, true/false questions, and short-answer questions. After completion, the system evaluates responses, calculates scores, highlights weak topics, and provides detailed explanations.

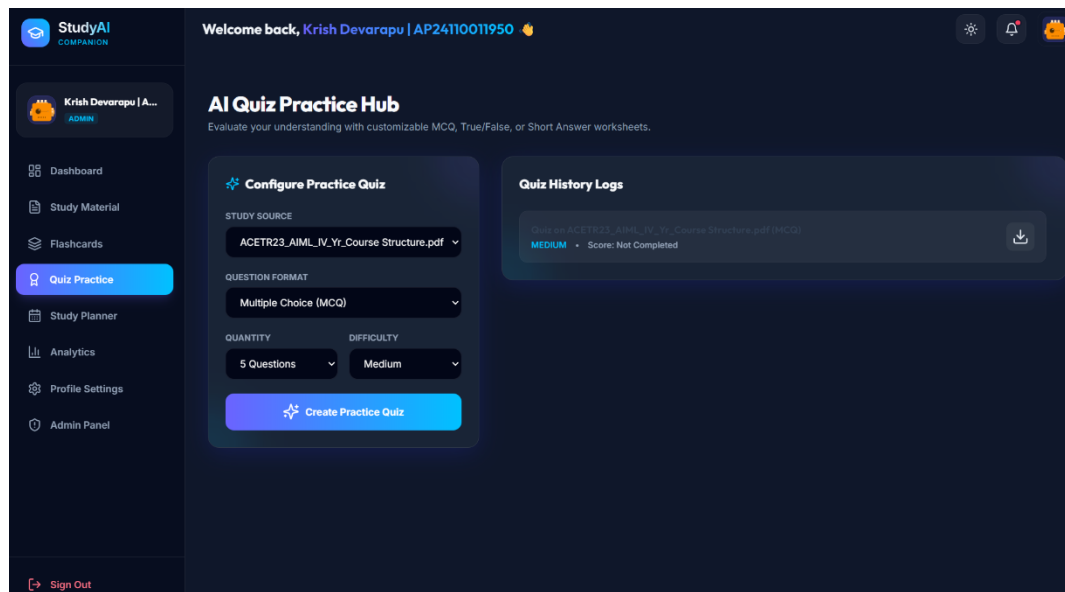


Figure 8.5: AI Quiz Generation

Personalized Study Schedule

The Study Schedule module generates a personalized 7-day learning plan based on uploaded materials and quiz performance. It recommends daily study tasks, focus topics, and revision schedules to help students prepare effectively.

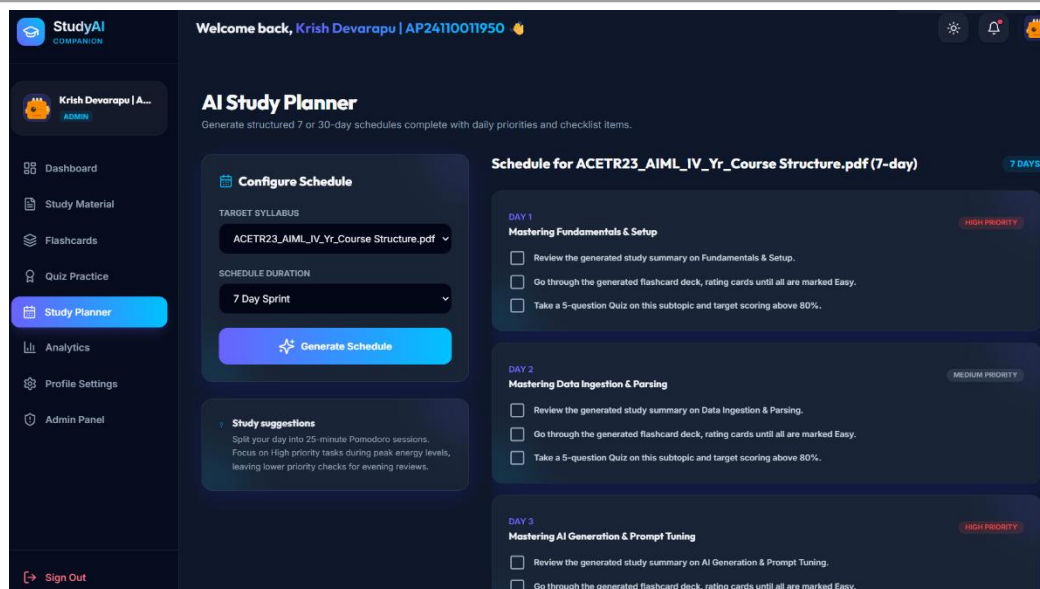


Figure 8.6: Personalized Study Schedule

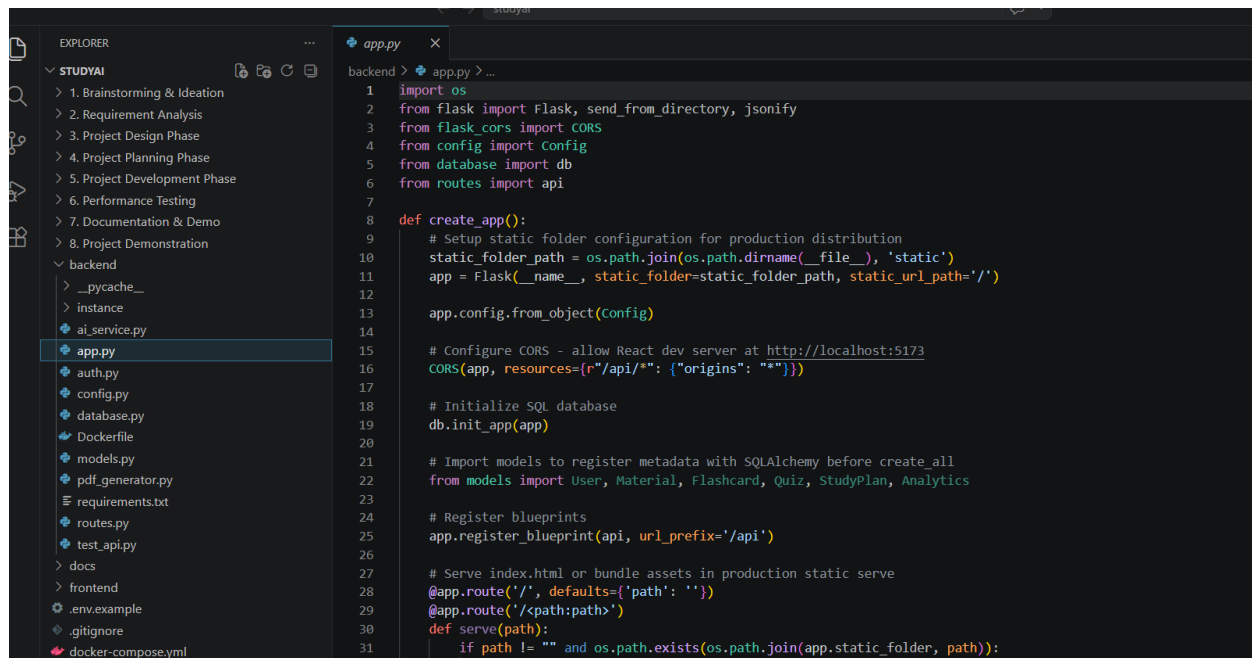
Milestone 3: Backend and Application Development

Introduction

The Backend and Application Development phase focuses on implementing the server-side logic, AI integration, database communication, and application workflow. During this milestone, the Flask backend, REST APIs, Firebase integration, and Groq AI services were developed to process study materials, generate AI-powered educational content, and manage application data securely.

Activity 3.1: Writing the Main Application Logic

The main application logic coordinates communication between the React frontend, Flask backend, Firebase database, and Groq AI services. It processes uploaded study materials, validates user requests, manages AI operations, and returns generated summaries, flashcards, quizzes, and personalized study schedules through REST APIs.



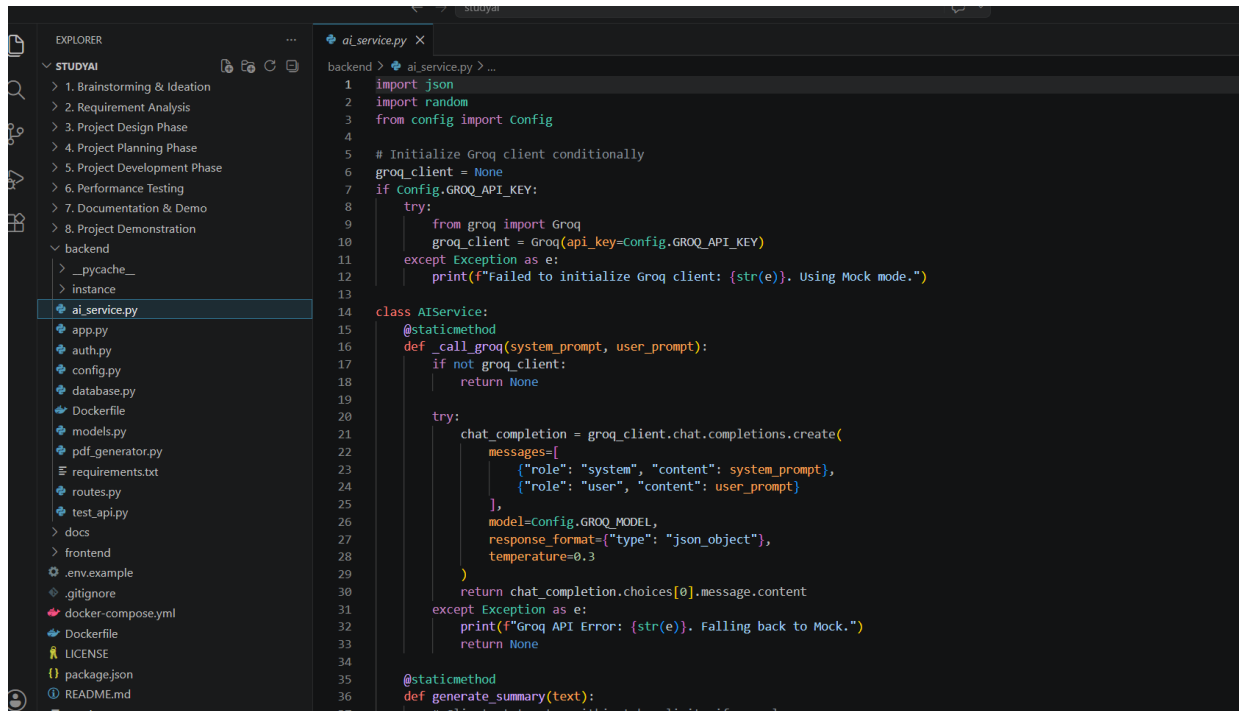
```

1 import os
2 from flask import Flask, send_from_directory, jsonify
3 from flask_cors import CORS
4 from config import Config
5 from database import db
6 from routes import api
7
8 def create_app():
9     # Setup static folder configuration for production distribution
10    static_folder_path = os.path.join(os.path.dirname(__file__), 'static')
11    app = Flask(__name__, static_folder=static_folder_path, static_url_path='/')
12
13    app.config.from_object(Config)
14
15    # Configure CORS - allow React dev server at http://localhost:5173
16    CORS(app, resources={r"/api/*": {"origins": "*"}})
17
18    # Initialize SQL database
19    db.init_app(app)
20
21    # Import models to register metadata with SQLAlchemy before create_all
22    from models import User, Material, Flashcard, Quiz, StudyPlan, Analytics
23
24    # Register blueprints
25    app.register_blueprint(api, url_prefix='/api')
26
27    # Serve index.html or bundle assets in production static serve
28    @app.route('/', defaults={'path': ''})
29    @app.route('/<path:path>')
30    def serve(path):
31        if path != "" and os.path.exists(os.path.join(app.static_folder, path)):
32            return send_from_directory(app.static_folder, path)
  
```

Figure 9.1: Main Backend Application Logic (app.py)

Activity 3.2: AI Study Summary Integration

The StudyAI application integrates the Groq API using the Llama 3.3 70B Versatile model to generate intelligent study content. The backend sends uploaded study material to the AI model, which returns structured summaries, flashcards, quizzes, and personalized study plans, improving students' learning experience.



```

1  import json
2  import random
3  from config import Config
4
5  # Initialize Groq client conditionally
6  groq_client = None
7  if Config.GROQ_API_KEY:
8      try:
9          from groq import Groq
10         groq_client = Groq(api_key=Config.GROQ_API_KEY)
11     except Exception as e:
12         print(f"Failed to initialize Groq client: {str(e)}. Using Mock mode.")
13
14 class AIService:
15     @staticmethod
16     def _call_groq(system_prompt, user_prompt):
17         if not groq_client:
18             return None
19
20         try:
21             chat_completion = groq_client.chat.completions.create(
22                 messages=[
23                     {"role": "system", "content": system_prompt},
24                     {"role": "user", "content": user_prompt}
25                 ],
26                 model=Config.GROQ_MODEL,
27                 response_format={"type": "json_object"},
28                 temperature=0.3
29             )
30             return chat_completion.choices[0].message.content
31         except Exception as e:
32             print(f"Groq API Error: {str(e)}. Falling back to Mock.")
33             return None
34
35     @staticmethod
36     def generate_summary(text):

```

Figure 9.2: AI Study Content Generation

Description

The backend architecture enables seamless communication between the frontend, database, and AI services. The Flask application manages API requests, performs AI processing, stores study data securely, and delivers personalized educational content efficiently.

Milestone 4: Frontend Development

Introduction

The Frontend Development phase focuses on designing and implementing a modern, responsive, and user-friendly interface for the StudyAI application. The frontend was developed using React.js to provide students with an interactive platform for uploading study materials, generating AI-powered summaries, creating flashcards, taking quizzes, viewing analytics, and managing personalized study schedules. The interface ensures smooth navigation, responsive layouts, and seamless communication with the Flask backend.

Activity 4.1: Designing and Developing the User Interface

The StudyAI user interface was developed using React.js with a clean and responsive design. The application provides a structured navigation menu, dashboard, study material management, AI summary generation, flashcards, quiz practice, analytics, and study planner modules. The consistent layout improves usability and allows students to access all learning features from a single platform.

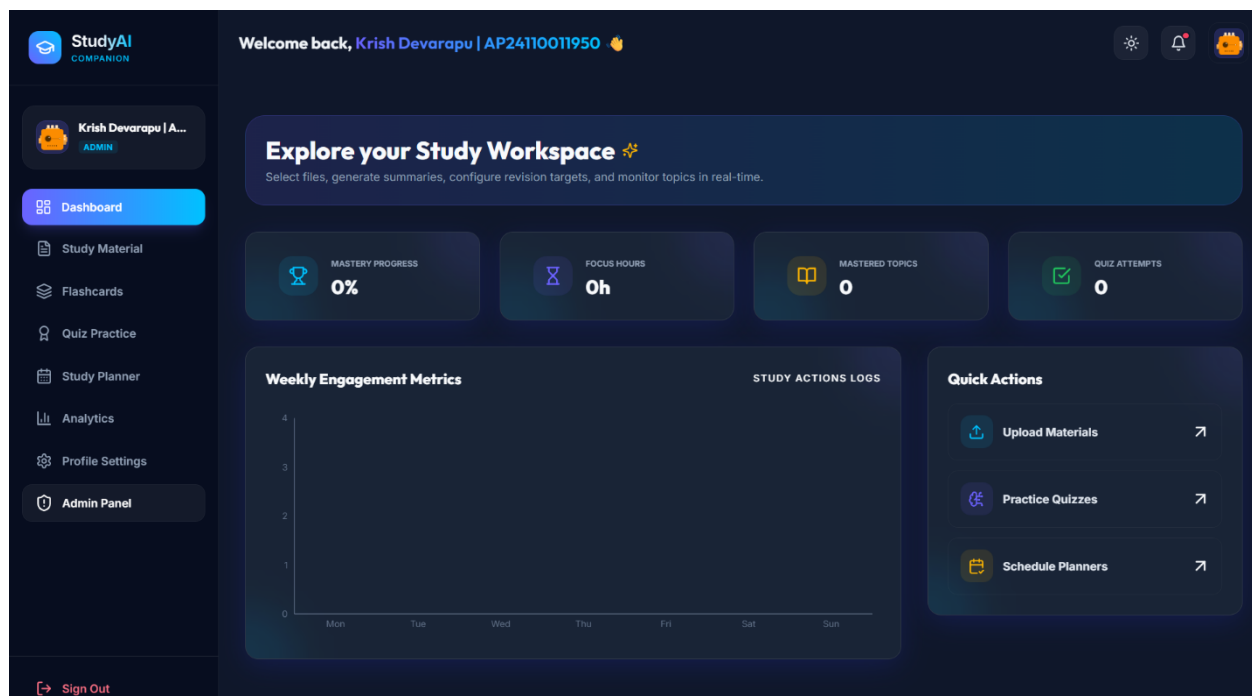


Figure 10.1: StudyAI Dashboard

Activity 4.2: Creating Dynamic Content Rendering with React

React components dynamically render study materials, AI-generated summaries, flashcards, quizzes, analytics, and personalized study schedules without requiring full page reloads. The frontend communicates with the Flask backend through REST APIs to display updated information instantly, providing a smooth and interactive learning experience.

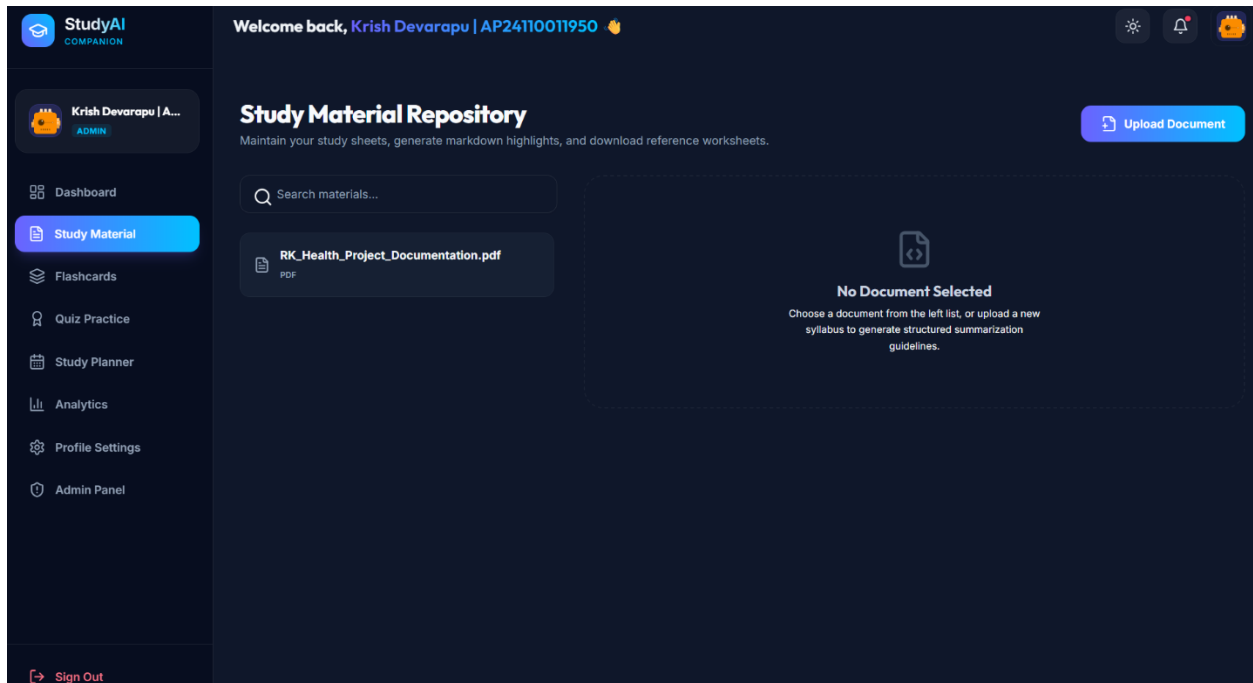


Figure 10.2: Study Material Management

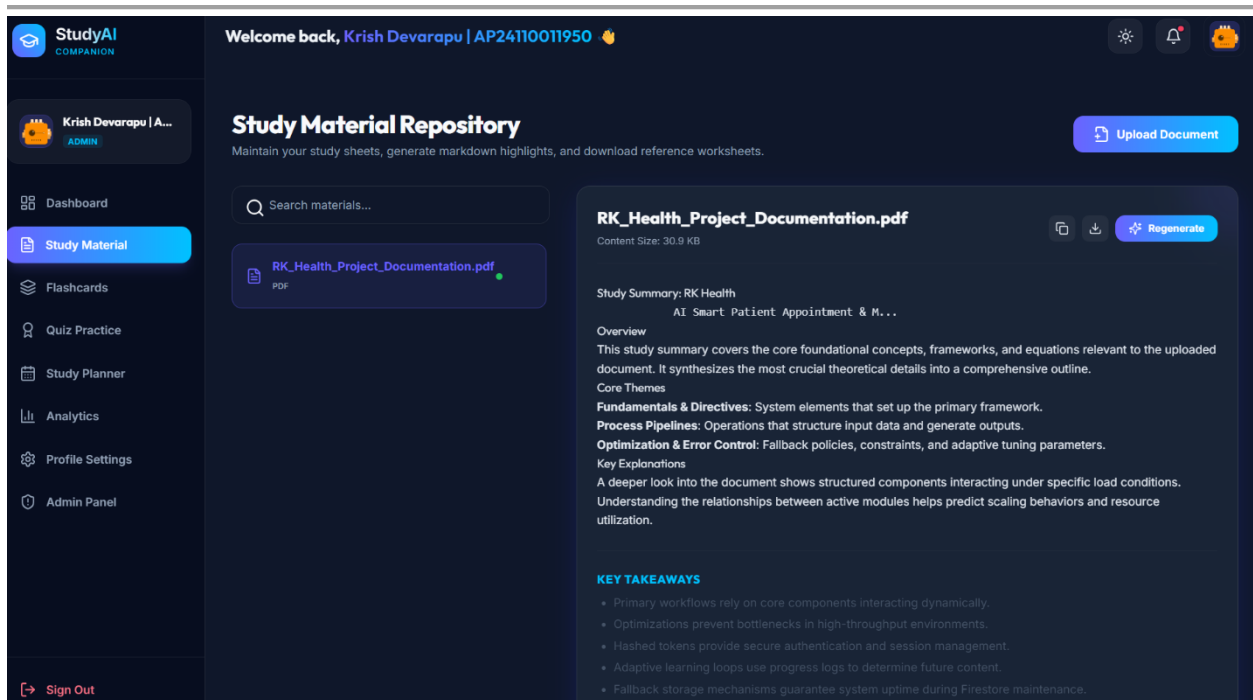


Figure 10.3: AI Summary Generation

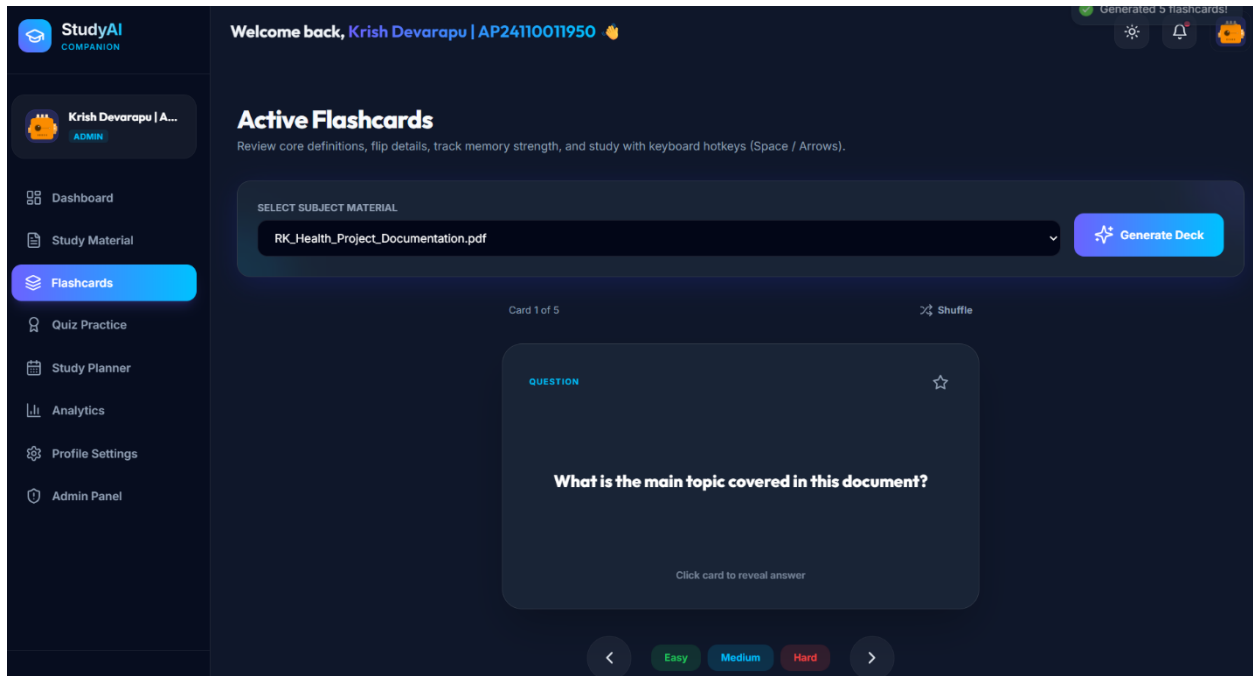


Figure 10.4: AI Flashcard Generation

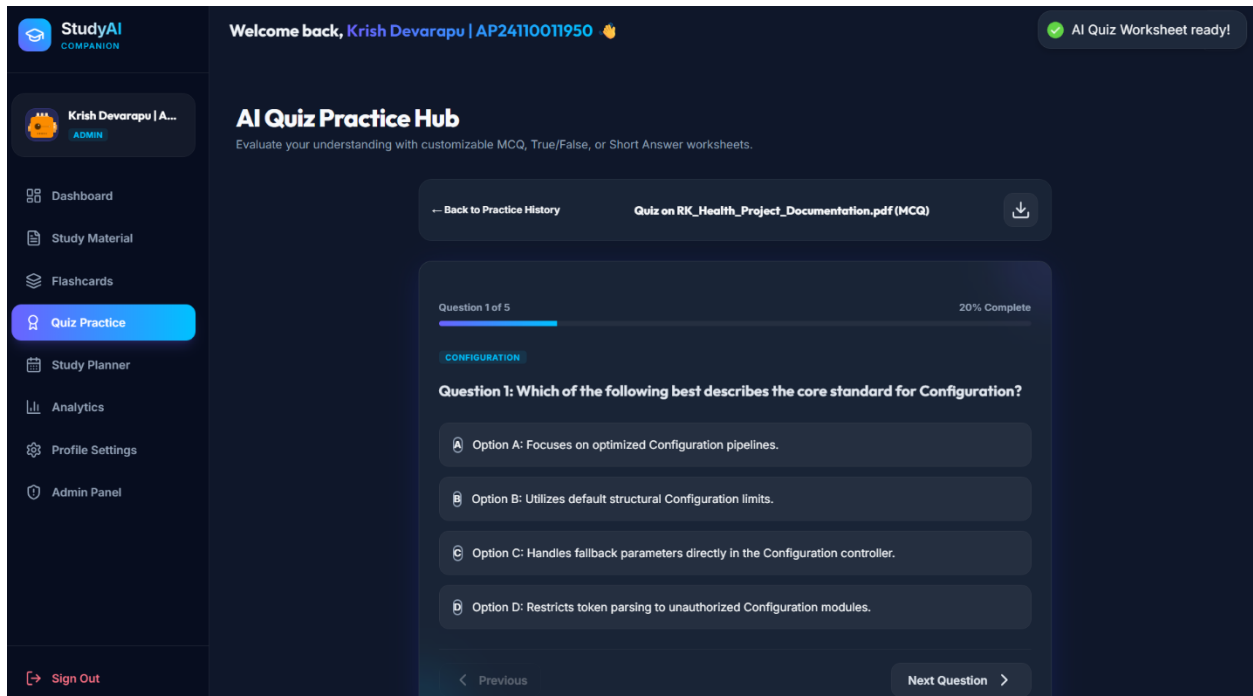
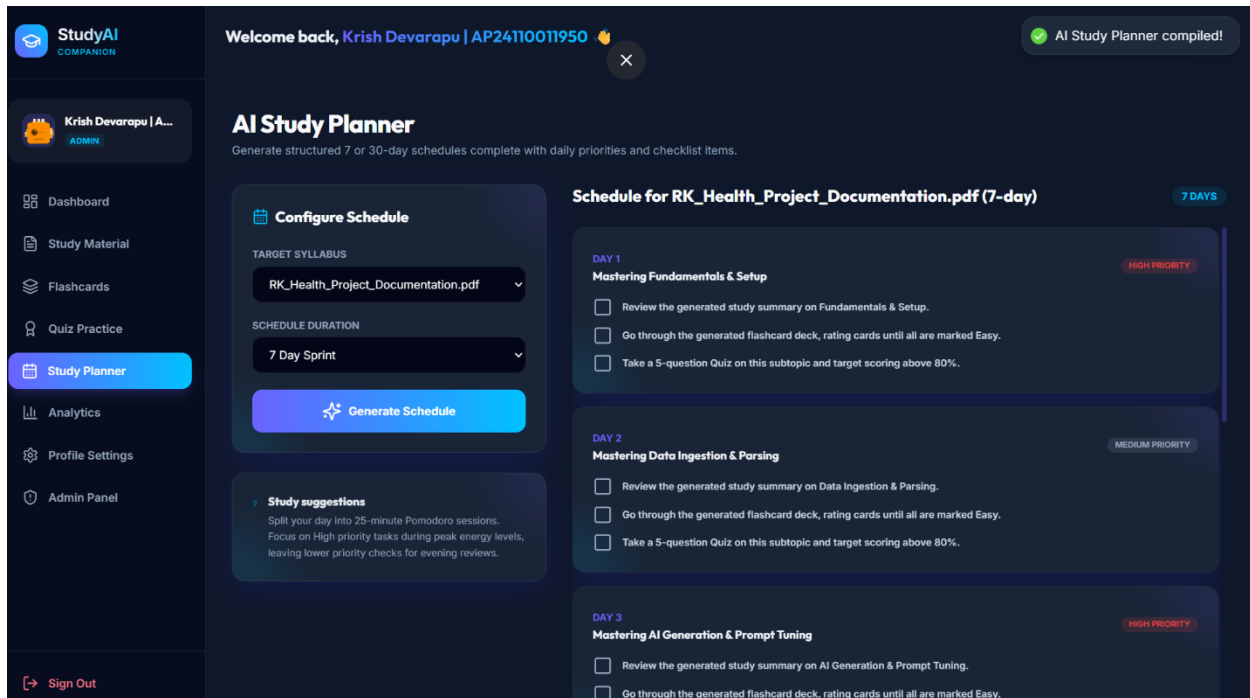


Figure 10.5: AI Quiz Interface



The screenshot displays the 'AI Study Planner' interface. At the top, a welcome message for 'Krish Devarapu' is shown. A sidebar on the left contains navigation links: Dashboard, Study Material, Flashcards, Quiz Practice, Study Planner (highlighted), Analytics, Profile Settings, and Admin Panel. The main content area is titled 'AI Study Planner' and includes a 'Configure Schedule' section with a dropdown for 'RK_Health_Project_Documentation.pdf' and a '7 Day Sprint' duration. A 'Generate Schedule' button is present. Below this, 'Study suggestions' are provided. The right side of the interface shows a 'Schedule for RK_Health_Project_Documentation.pdf (7-day)' with a '7 DAYS' badge. It lists three days of tasks: Day 1 (Mastering Fundamentals & Setup, High Priority), Day 2 (Mastering Data Ingestion & Parsing, Medium Priority), and Day 3 (Mastering AI Generation & Prompt Tuning, High Priority). Each day has a list of tasks with checkboxes.

Figure 10.6: Personalized Study Schedule

Description

The React-based frontend dynamically communicates with the Flask backend to display AI-generated educational content in real time. Students can upload study materials, generate summaries, practice with flashcards, take quizzes, analyze performance, and follow personalized study schedules through a responsive interface. The application delivers a seamless and engaging learning experience across different devices.

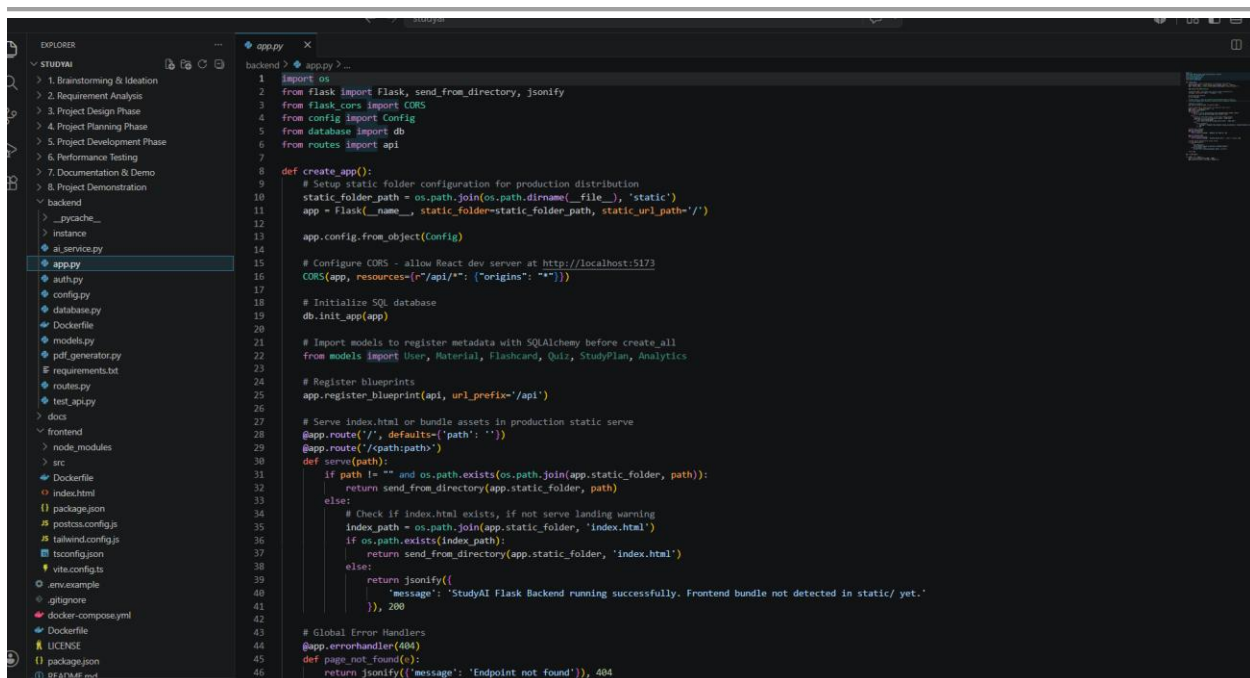
Milestone 5: Deployment

Introduction

The Deployment phase focuses on preparing the StudyAI application for production and validating its functionality in a live environment. During this milestone, the frontend, backend, Firebase services, and Groq AI integration were configured, tested, and deployed to ensure reliable performance. The deployed application enables students to access AI-powered study tools through a web browser with secure communication between all application components.

Activity 5.1: Preparing the Application for Deployment

The StudyAI project was prepared for deployment by organizing the project structure, configuring environment variables, installing required dependencies, and verifying communication between the React frontend, Flask backend, Firebase services, and Groq AI platform. Proper configuration ensures smooth application execution and simplifies future maintenance.



The screenshot shows the VS Code interface with the Explorer panel on the left displaying the project structure. The main editor shows the `app.py` file. The Explorer panel lists the following folders and files:

- STUDYAI
 - 1. Brainstorming & Ideation
 - 2. Requirement Analysis
 - 3. Project Design Phase
 - 4. Project Planning Phase
 - 5. Project Development Phase
 - 6. Performance Testing
 - 7. Documentation & Demo
 - 8. Project Demonstration
 - backend
 - __pycache__
 - instance
 - ai.service.py
 - app.py**
 - auth.py
 - config.py
 - database.py
 - Dockerfile
 - models.py
 - pdf_generator.py
 - requirements.txt
 - routes.py
 - test_api.py
 - docs
 - frontend
 - node_modules
 - src
 - Dockerfile
 - index.html
 - package.json
 - postcss.config.js
 - tailwind.config.js
 - tsconfig.json
 - vite.config.ts
 - env.example
 - .gitignore
 - docker-compose.yml
 - Dockerfile
 - LICENSE
 - package.json
 - README.md

The `app.py` file content is as follows:

```
1 import os
2 from flask import Flask, send_from_directory, jsonify
3 from flask_cors import CORS
4 from config import Config
5 from database import db
6 from routes import api
7
8 def create_app():
9     # Setup static folder configuration for production distribution
10     static_folder_path = os.path.join(os.path.dirname(__file__), 'static')
11     app = Flask(__name__, static_folder=static_folder_path, static_url_path='/')
12
13     app.config.from_object(Config)
14
15     # Configure CORS - allow React dev server at http://localhost:5173
16     CORS(app, resources={r"/api/*": {"origins": "*"}})
17
18     # Initialize SQL database
19     db.init_app(app)
20
21     # Import models to register metadata with SQLAlchemy before create_all
22     from models import User, Material, Flashcard, Quiz, StudyPlan, Analytics
23
24     # Register blueprints
25     app.register_blueprint(api, url_prefix="/api")
26
27     # Serve index.html or bundle assets in production static serve
28     @app.route("/", defaults={"path": ""})
29     @app.route("/<path:path>")
30     def serve(path):
31         if path != "" and os.path.exists(os.path.join(app.static_folder, path)):
32             return send_from_directory(app.static_folder, path)
33         else:
34             # Check if index.html exists, if not serve landing warning
35             index_path = os.path.join(app.static_folder, 'index.html')
36             if os.path.exists(index_path):
37                 return send_from_directory(app.static_folder, 'index.html')
38             else:
39                 return jsonify({
40                     'message': 'StudyAI Flask Backend running successfully. Frontend bundle not detected in static/ yet.'
41                 }), 200
42
43     # Global Error Handlers
44     @app.errorhandler(404)
45     def page_not_found(e):
46         return jsonify({'message': 'Endpoint not found'}), 404
```

Figure 11.1: StudyAI Project Structure

Activity 5.2: Deploying the Application

The complete project source code was maintained using GitHub for version control and collaborative development. The application components were deployed to enable real-time access to AI-powered learning features. Deployment ensured secure communication between the frontend, backend, Firebase cloud services, and Groq AI.

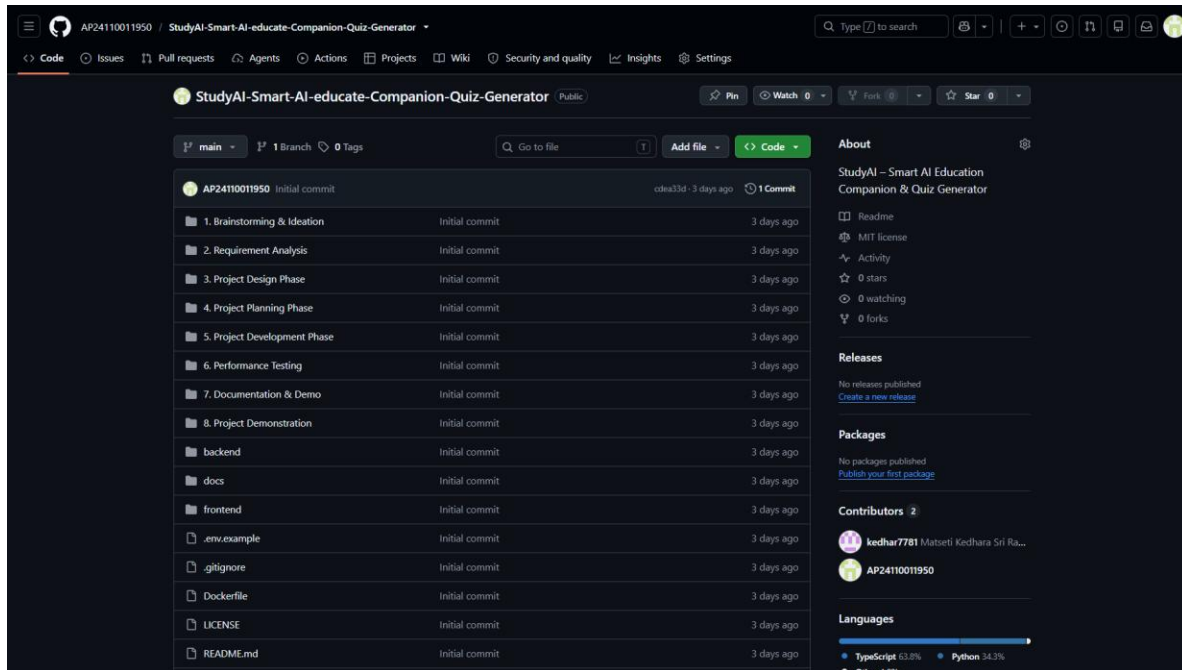


Figure 11.2: GitHub Repository Deployment

Backend Deployment

The Flask backend was configured to manage API requests, AI processing, study material handling, Firebase communication, and cloud services. Backend modules coordinate AI-powered summary generation, flashcard creation, quiz generation, analytics, and personalized study planning while ensuring secure data processing.

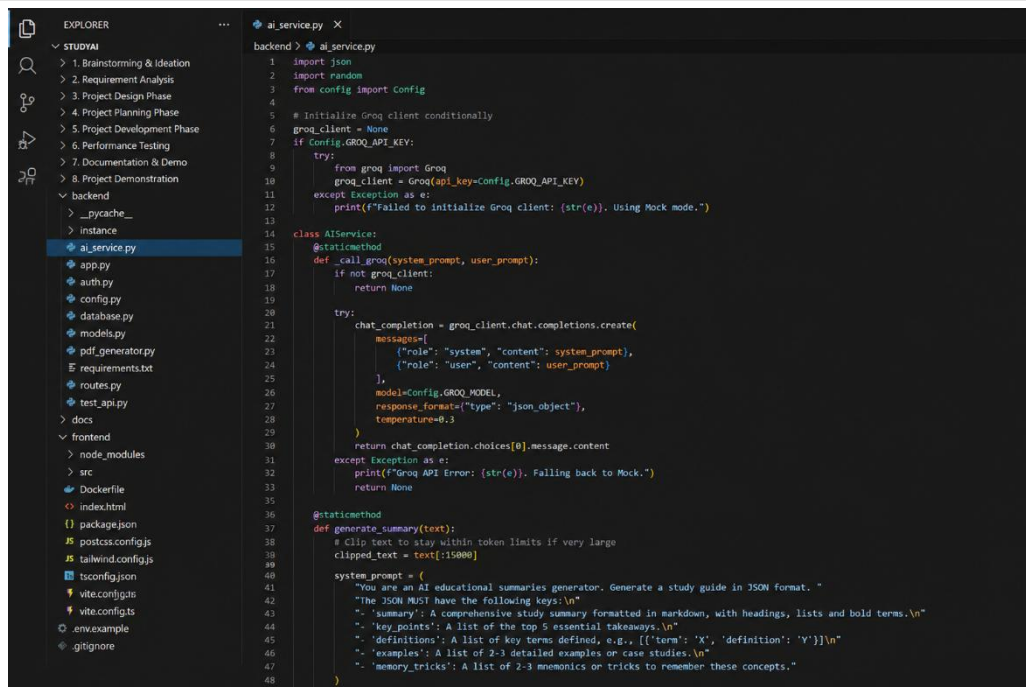


Figure 11.3: Backend AI Service Integration

Activity 5.3: Validate Deployment and Test Live Workflow

Comprehensive testing was performed after deployment to verify study material uploads, AI summary generation, flashcard creation, quiz generation, analytics, and personalized study schedule generation. The deployed application successfully completed functional testing and demonstrated stable performance across all major learning modules.

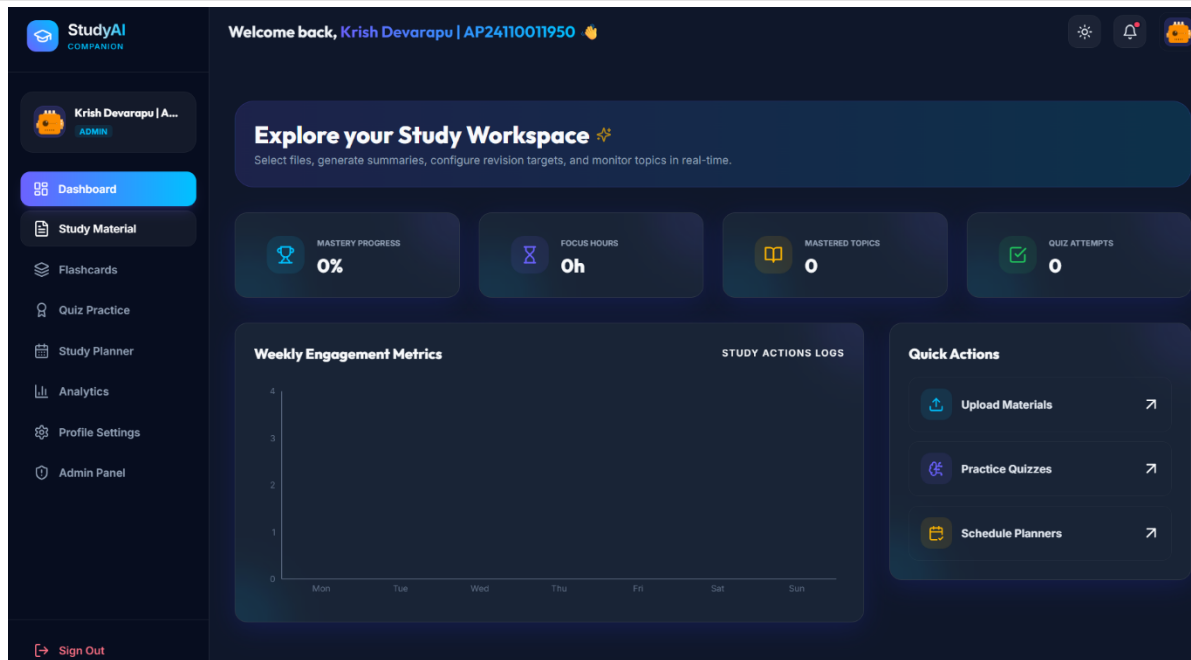


Figure 11.4: Live Application Validation

Description

The deployment process ensured successful integration of the React frontend, Flask backend, Firebase cloud platform, and Groq AI services. Functional testing confirmed reliable application performance, secure data communication, responsive user interaction, and accurate AI-generated educational content. The deployed system provides students with a stable and intelligent learning platform.

Conclusion

The StudyAI project was successfully developed as an AI-powered educational platform that helps students improve their learning through intelligent study tools. The application enables users to upload study materials, generate AI-powered summaries, create flashcards, practice quizzes, analyze learning performance, and receive personalized study schedules through a single web-based interface.

The project integrates React.js, Python Flask, Firebase, and the Groq AI platform to provide fast, accurate, and interactive educational content. Throughout the project, all major software development phases—including requirement analysis, system design, implementation, testing, deployment, and documentation—were successfully completed.

The application demonstrates how Generative AI can simplify exam preparation and personalized learning while providing an efficient, responsive, and user-friendly experience. Future enhancements may include collaborative learning, multilingual support, mobile application deployment, offline learning capabilities, and advanced AI-powered learning analytics.

Activity 6.1: Final Documentation, Testing, and Review

The StudyAI project was successfully completed following a structured software development lifecycle that included requirement analysis, system design, implementation, testing, deployment, and documentation. All functional modules were verified, screenshots were documented, deployment was validated, and the complete project workflow was reviewed to ensure accuracy, consistency, and readiness for project evaluation.